



Escuela de ingeniería en computación
Ingeniería en computación
IC6821 - Diseño de Software

Patrón Decorator

Implementación - Tarea corta

Dayron Josué Porras Pérez
dayporras@estudiantec.cr
2023124693

Ian Sebastian Ugalde Muñoz
i.ugalde.1@estudiantec.cr
2024124076

Marcelo Gómez Cordero
mar_gomez@estudiantec.cr
2022136492

Javier Lee Liang
javi0409@estudiantec.cr
2020184824

José Emilio Alvarado Méndez
jos.alvarado@estudiantec.cr
2022163260

San José, Costa Rica
Junio, 2026

Índice general

1. Introducción	2
2. Diagrama de Clases Completo de la Solución	3
2.1. Clases pertenecientes al patrón Decorator	3
3. Descripción de la Implementación del Patrón	5
3.1. Contexto	5
3.2. Estructura	5
3.3. Ventajas Obtenidas	6
4. Repositorio del Proyecto	7
5. Conclusiones	8

Capítulo 1

Introducción

Este documento presenta el diseño e implementación del patrón de diseño *Decorator* dentro de la solución desarrollada para el proyecto final del curso de Patrones de Diseño.

La solución consiste en un IDE de escritorio para Windows, desarrollado en C# con .NET 8 y Windows Forms, orientado a la entrega y revisión de ejercicios de programación en Python. El sistema contempla autenticación de usuarios, gestión de tareas y grupos de trabajo, ejecución de código en terminal integrada y, como elemento central de este documento, un mecanismo de **firma e integridad de archivos** implementado mediante el patrón Decorator.

Se describe la estructura general del sistema, las clases que participan en el patrón, el funcionamiento del mismo y los resultados obtenidos al integrarlo con la interfaz de usuario del IDE.

Capítulo 2

Diagrama de Clases Completo de la Solución

En la figura 2.1 se muestra el diagrama de clases donde se implementa el patrón decorator en relación al proyecto principal.

2.1. Clases pertenecientes al patrón Decorator

Para facilitar la identificación del patrón Decorator dentro del diagrama, las clases que lo conforman se resaltan en color **naranja** dentro del paquete `ScriptDecoratorApp`:

- **IScript** (Component): interfaz base que define el contrato con los métodos `GetPath()` y `GetText()`.
- **ConcreteScript** (ConcreteComponent): implementación concreta de **IScript**; almacena la ruta y el contenido textual de un script Python.
- **ScriptDecorator** (Decorator): clase abstracta que implementa **IScript** y mantiene una referencia interna al componente envuelto (`_inner`). Delega las llamadas por defecto al componente envuelto.
- **SignedScript** (ConcreteDecorator): extiende **ScriptDecorator** añadiendo la capacidad de **firmar** el contenido del script con SHA-256 y **verificar** su integridad sin modificar ninguna clase existente.

Estas clases permiten agregar la responsabilidad de firma e integridad a cualquier objeto que implemente **IScript** de manera dinámica, sin alterar la implementación original del componente.

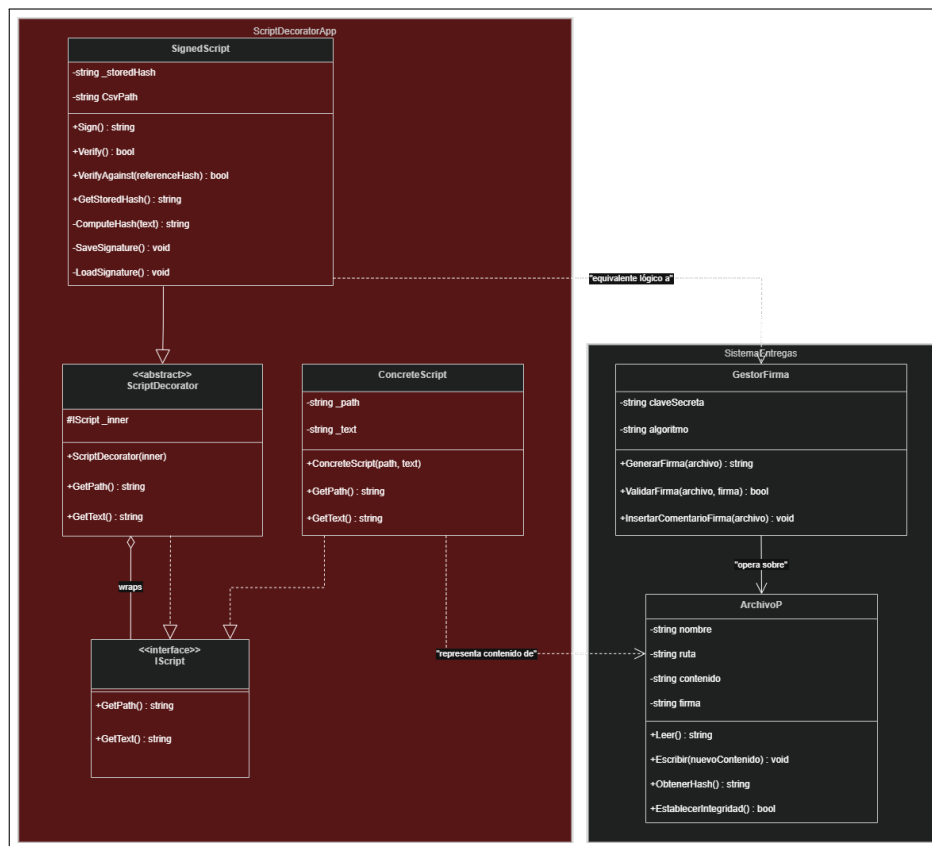


Figura 2.1: Diagrama de clases completo de la solución, organizado por paquetes. Las clases del patrón Decorator se resaltan en color rojizo dentro del paquete **ScriptDecoratorApp**.

Capítulo 3

Descripción de la Implementación del Patrón

3.1. Contexto

En el sistema de entregas, el usuario edita archivos Python dentro del IDE. Un requisito clave es garantizar la **integridad** del archivo: detectar si fue modificado fuera del entorno de entrega después de haber sido firmado. Implementar esta funcionalidad directamente en la clase `ArchivoP` o en el formulario IDE habría acoplado la lógica de criptografía al modelo de dominio y a la vista. El patrón Decorator resuelve esto de manera elegante.

3.2. Estructura

La implementación se compone de cuatro elementos ubicados en el espacio de nombres `ScriptDecoratorApp`:

1. **IScript** — interfaz *Component* con dos métodos: `GetPath()` y `GetText()`.
2. **ConcreteScript** — *ConcreteComponent* que recibe la ruta y el texto del archivo en su constructor.
3. **ScriptDecorator** — clase abstracta *Decorator* con campo protegido `_inner` : `IScript`; delega por defecto.
4. **SignedScript** — *ConcreteDecorator* que agrega:
 - `Sign()`: calcula el hash SHA-256 del contenido y lo persiste en `signatures.csv`.
 - `Verify()`: carga la firma desde el CSV y compara contra el hash actual del contenido.
 - `VerifyAgainst(hash)`: permite comparar contra un hash provisto externamente.

- `GetStoredHash()`: retorna la firma almacenada, cargándola del CSV si es necesario.

3.3. Ventajas Obtenidas

- **Principio Abierto/Cerrado**: la funcionalidad de firma se añade sin modificar `ConcreteScript`, `ArchivoP` ni ninguna clase existente.
- **Responsabilidad única**: la criptografía queda encapsulada exclusivamente en `SignedScript`.
- **Extensibilidad**: es posible agregar nuevos decoradores (por ejemplo, `CompressedScript` o `EncryptedScript`) apilándolos sobre `ConcreteScript` sin tocar el código actual.
- **Bajo acoplamiento**: el formulario IDE solo conoce la interfaz `IScript`; puede sustituirse cualquier decorador sin afectar la vista.
- **Reutilización**: `SignedScript` puede reutilizarse en cualquier parte del sistema que opere con objetos `IScript`, independientemente del contexto de interfaz gráfica.

Capítulo 4

Repositorio del Proyecto

El código fuente completo, junto con el ejecutable compilado y funcional, se encuentra disponible en el siguiente repositorio de GitHub:

`https://github.com/DayRPK25/Decorator`

El repositorio incluye:

- Código fuente completo del proyecto en C# (.NET 8, Windows Forms).
- Paquete `ScriptDecoratorApp` con la implementación del patrón Decorator.
- Ejecutable compilado en `bin/Release/net8.0-windows/`.
- Archivo `signatures.csv` generado al firmar scripts.
- Documentación del proyecto (este documento).

Capítulo 5

Conclusiones

La implementación del patrón Decorator en **SignedScript** permitió agregar la capacidad de firma SHA-256 a cualquier objeto **IScript** de forma transparente y sencilla, sin alterar las clases existentes del dominio (**ArchivoP**, **GestorFirma** ni el formulario **IDE**).

El uso del patrón favoreció el cumplimiento del *Principio Abierto/Cerrado*: el sistema está abierto para extensión (nuevos decoradores) y cerrado para modificación (las clases base no cambian).

La solución basada en Decorator resultó más flexible que una alternativa por herencia, ya que permite combinar o agregar una mayor cantidad de decoradores en tiempo de ejecución, dando así un mayor dinamismo.

La separación de responsabilidades facilita el mantenimiento, ya que la lógica criptográfica está contenida en una sola clase (**SignedScript**), lo que simplifica las pruebas que podrían realizarse y futuras modificaciones del algoritmo de firma.

El proyecto permitió entender de mejor forma la aplicación práctica de patrones sobre un desarrollo de software orientado a objetos, especialmente en contextos donde la extensión de comportamiento debe realizarse sin comprometer la estabilidad y eficacia del código existente.